



lab3

lab3-1

设计简单的有限状态机

一、使用enum语法设计FSM

- 设计思路：
 - i. 采用enum枚举方法实现有限状态机，先使用enum定义一个枚举类型，由于该状态机只有四个状态，该类型宽度设为**2位**即可，即使用两位数据表示四个状态
 - ii. {S0,S1,S2,S3} 一次性枚举常量，分别代表四个状态
 - iii. **异步复位**，当复位信号处于低电平，直接将状态置S0
 - iv. **同步更新**，当时钟上沿到来时，根据当前状态和输入X，通过状态表决定跳转到哪个状态
- 代码实现：

```
module FSM(  
    input rstn,  
    input clk,  
    input a,  
    input b,  
    output [1:0] state  
);  
  
typedef enum logic [1:0] {S0, S1, S2, S3} fsm_state;  
fsm_state state_reg;  
always @(posedge clk or negedge rstn) begin  
    if(~rstn) begin  
        state_reg <= S0;  
    end  
    else begin  
        case(state_reg)  
            S0: if(a == 0 && b == 1) state_reg <= S0;  
                else if(a == 1) state_reg <= S1;  
                else state_reg <= S0;  
        endcase  
    end  
end
```

```

S1: if(a == 0 && b == 1) state_reg <= S0;
    else if(a == 1) state_reg <= S2;
    else state_reg <= S1;

S2: if(a == 0 && b == 1) state_reg <= S0;
    else if(a == 1) state_reg <= S3;
    else state_reg <= S2;

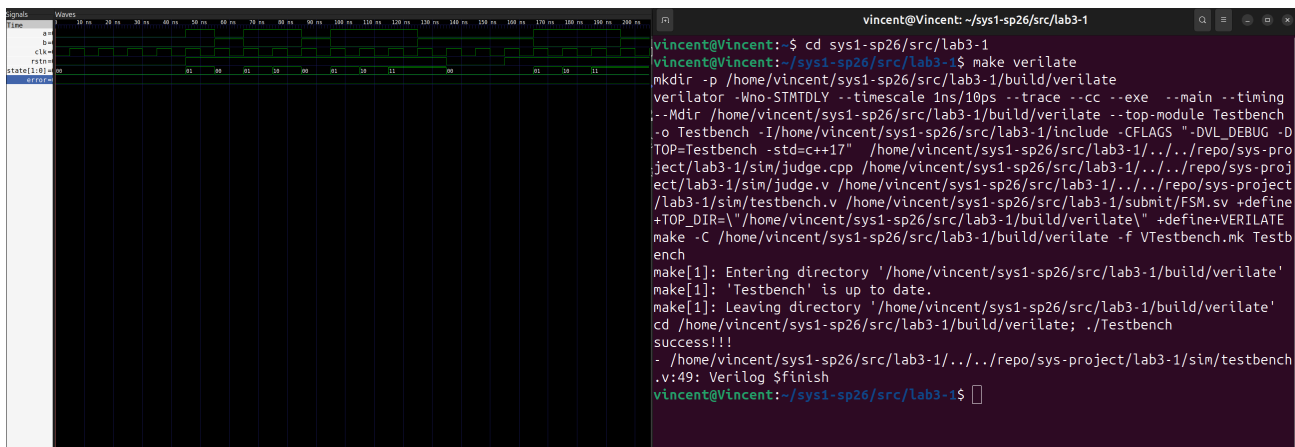
S3: state_reg <= S3;

    endcase
end

assign state = state_reg;
endmodule

```

- **仿真截图 & 仿真通过截图:**



二、思考题

1. 一、enum + case 的编程范式和数组查表的编程范式之间的优劣

- **可读性：**`enum + case`的编程范式更加直观体现**状态之间的转换**，而**查表法**数据需要转换才能理解含义，看起来不直观
- **易修改：**`enum + case`不易修改，每次需要修改所有的switch分支，容易遗漏，**数组查表**只需要扩展表格，不用修改核心循环，更好维护和修改
- **适用范围：**`enum + case`适合状态少并转换关系复杂的FSM，**数组查表**适合状态

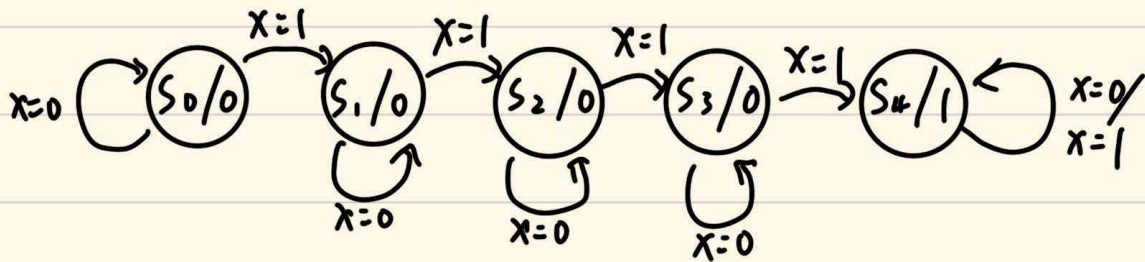
多，转换规则简单的FSM

2. 二、要求有限状态机的状态图和状态转移表

- 状态表：

一、状态图：

$S_0: 0 \uparrow 1$ $S_1: 1 \uparrow 1$ $S_2: 2 \uparrow 1$ $S_3: 3 \uparrow 1$ $S_4: 4 \uparrow 1$



- 状态转移表：

present state	X	Next state	Output
S0	0	S0	0
S0	1	S1	0
S1	0	S1	0
S1	1	S2	0
S2	0	S2	0
S2	1	S3	0
S3	0	S3	0
S3	1	S4	1
S4	0	S4	1
S4	1	S4	1

3. 有限状态机电路实现的不足：

```
always@(posedge clk)begin
    if(~rstn) state <= 2'b01;
    else state[1:0] <= state[0:1];
end
```

- **实现功能：**初始状态为 01，每个时钟上沿到来时，将state实现从 01 和 10 之间的来回切换
- **不足之处1：**该写法为同步复位，只有时钟上沿时 rstn 为低电平才会被成功初始化，如果复位信号很短，在时钟上沿前就恢复高电平，就可能无法成功复位，使得整个触发器保持在非法状态，建议改成异步触发 `alwys@(posedge clk or negedge rstn)`
- **不足之处2：**缺少外部控制，该状态机只能实现两个状态不受任何外部输入控制，实际作用很小，不如直接使用clk信号
- **不足之处3：**设计时没有使得4个状态都合法，一旦出现波动使得系统出现 00 和 11，系统将无法修复，所有逻辑功能全部错误，应该加入default分支直接回到初始状态

lab3-2

计数器/定时器设计与应用

一、实现计数器模块Cnt.sv

- **设计思路：**
 - 模数可设置：**在调用时 BASE 可设置，默认为10，即计数值从0到9，当计数为9且有低位输入 low_co 时，产生进位信号并将计数值归“零”（归到基础值）
 - 进位信号：**co 作为进位信号，只有计数为 BASE-1 且同时有 low_co 和 en 时产生。
 - 使能信号：**en 作为全局使能信号，当en=1时，计数器才开始接受 low_co, high_rst
 - 低位输入：**low_co 作为低位的输入/进位信号，当时钟上沿到来时，计数器只有在 en && low_co 时递增
 - 高层复位：**high_rst 用于非模数溢出时强制归零，而不是回到初始值，为了BCD码计数器的实现设计的控制信号
 - 底层Cnt复位：**低位的Cnt有两种复位可能，一种是达到自身BASE，一种为达到整个计数器的BASE通过上层复位信号复位
- **代码实现：**

```

module Cnt #(
    parameter BASE = 10,
    parameter INITIAL = 0
) (
    input en,
    input clk,
    input rstn,
    input low_co,
    input high_rst,
    output co,
    output reg [3:0] cnt
);

assign co = (cnt == BASE - 1) && en && low_co;

always @(posedge clk or negedge rstn) begin
    if(!rstn) begin
        cnt <= INITIAL;
    end else if(en && high_rst) begin
        cnt <= 0;
    end else if(en && low_co) begin
        if(cnt == BASE - 1) begin
            cnt <= 0;
        end else begin
            cnt <= cnt + 4'b1;
        end
    end
end
endmodule

```

二、设计24-BCD码计数器Cnt2num.sv

- 设计思路：

- 实现框架:**通过实例化两个Cnt.sv来实现模24的BSC码计数器，输出八位的BCD码(高四位表示十位上数字的二进制，低四位表示个位上数字的二进制),分别为两个Cnt的输出
- 全局使能信号:**使能信号作用在全局，只用en=1，计数器才会出现除复位以外的状态改变
- 进位信号:**基础仍然为 `cnt输出 = BASE-1 && en && low_co`，只不过在BCD码计数器

中，cnt输出应该分高位cnt和低位cnt，分别和BASE的高四位和第四位比较，

iv. **高层复位信号**：如果BASE的个位与低位Cnt的BASE不同，就需要 high_rst 强制将 Cnt置零

v. 高位Cnt的 low_co 为低位Cnt的 co

- **代码实现：**

```
module Cnt2num #(
    parameter BASE = 24,
    parameter INITIAL = 16
)(
    input en,
    input clk,
    input rstn,
    input high_rst,
    input low_co,
    output co,
    output [7:0] cnt
);

    localparam HIGH_BASE = 10;
    localparam LOW_BASE = 10;
    localparam HIGH_INIT = INITIAL/10;
    localparam LOW_INIT = INITIAL%10;
    localparam HIGH_CO = (BASE-1)/10;
    localparam LOW_CO = (BASE-1)%10;

    wire [3:0] cnt_high,cnt_low;
    wire low_co_to_high;
    wire inner_high_rst;
    assign cnt = {cnt_high, cnt_low};
    assign co = (cnt_high == 4'(HIGH_CO)) && (cnt_low == 4'(LOW_CO)) && en &&
low_co;
    assign inner_high_rst = high_rst || co;

    Cnt #(
        .BASE(HIGH_BASE),
        .INITIAL(4'(HIGH_INIT))
    ) high_cnt (
        .en(en),
```

```

        .clk(clk),
        .rstn(rstn),
        .high_rst(inner_high_rst),
        .low_co(low_co_to_high),
        .co(),
        .cnt(cnt_high)
    );

```

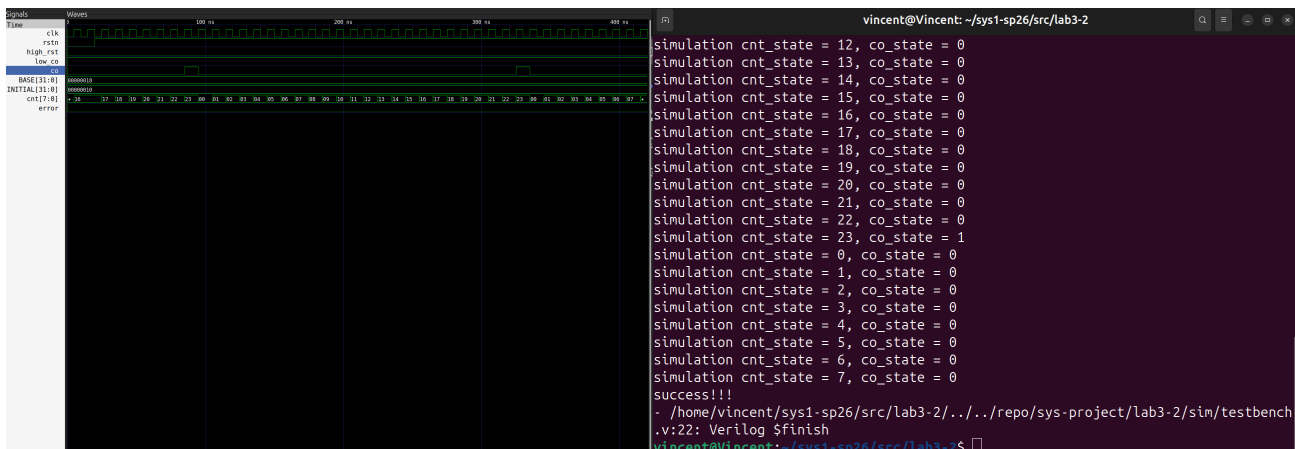
```

Cnt #(
    .BASE(LOW_BASE),
    .INITIAL(4'(LOW_INIT))
) low_cnt (
    .en(en),
    .clk(clk),
    .rstn(rstn),
    .high_rst(inner_high_rst),
    .low_co(low_co),
    .co(low_co_to_high),
    .cnt(cnt_low)
);

```

endmodule

• 仿真截图 & 仿真通过截图：



三、思考题

1. 使用Cnt2num.sv实现1234 BCD码计数器设计思路：

- 实例化两个Cnt2num.sv，分别处理高二位和低二位，高二位：BASE=13, INITIA

L=0,低二位: BASE=100,INITIAL=0

- 高两位的 low_co 为低两位的 co
- 此时已经形成了一个模1300的BCD码计数器，最后通过高层复位实现模1234
- 设置复位信号为，当高两位输出为12，低两位输出为33，且用使能信号，有低位输入时，给两个模块的 high_rst 同步施加一个脉冲，使得整个计数器复位

2. co,low_co,high_rst引脚的意义:

引脚	
co	进位输出：当本级计数器达到最大值（BASE-1）且 low_co 有效时，co 有效。用于连接上
low_co	低位进位输入：对于高位计数器，low_co来自低一级计数器的 co，决定本级是否递增。对
high_rst	上层强制复位：当整体计数值到达非低级计数器自然溢出的边界（如从23跳回0，而低位计

- **总结：**三个引脚将多个计数器有效连接，实现了多位的BCD码计数器。并通过 high_rst 信号，实现从只能为整数的BCD码计数器，到可以实现任何模数的BCD码计数器

lab3-3

乘法器的实现（multiplier）

- **设计思路：**
 - i. **设计架构：**状态机控制三个状态：
 - a. **IDLE:**检测到start后，初始化数据进入 WORK 状态，开始计算过程，而不接受到 start 信号就保持在 IDLE 状态
 - b. **WORK:**通过 work_cnt 控制计算过程循环32次，每次时钟周期完成一次移位相加步骤，并使得 work_cnt 递增，当 work_cnt 与位数相等时，跳转到 FINAL 状态，不然下个周期仍在 WORK 状态
 - c. **IDLE:**将乘积锁存到输出 product，并置 finish=1 持续一个周期，完成握手协议，之后回到 IDLE 等待下一次请求。
 - ii. **移位相加计算过程：**
 - a. 如果 **product_reg[0] == 1'b0**(即当前乘数最低位为0)，输出锁存器只需要整体右移一位，高位补零，**product_reg <= {1'b0, product_reg[PRODUCT_LEN-1:1]}**;
 - b. 如果 **product_reg[0] == 1'b1**(即当前乘数最低位为1)，输出锁存器高32位需要和背乘数相加，并整体右移一位，则结果分为两个部分：

- **高33位**（预留一位确保相加结果不会溢出）（**{1'b0, product_reg[PRODUCT_LEN-1:LEN]} + multiplicand_reg**）
- **低31位**，直接使用原锁存器从[LEN-1:1]的值（即右移后舍弃最后一位），
- **最后结果为****product_reg <= {(1'b0, product_reg[PRODUCT_LEN-1:LEN]} + multiplicand_reg), product_reg[LEN-1:1]}**;

- **代码实现：**

```
module Multiplier #(
    parameter LEN = 32
) (
    input clk,
    input rst,
    input [LEN-1:0] multiplicand,
    input [LEN-1:0] multiplier,
    input start,

    output [LEN*2-1:0] product,
    output finish
);

//乘机的位宽
localparam PRODUCT_LEN = LEN*2;

//被乘数寄存器
logic [LEN-1:0] multiplicand_reg;

//乘积寄存器
logic [PRODUCT_LEN-1:0] product_reg;

//计数到LEN需要多少个迭代
localparam CNT_NUM = LEN - 1;

//有限状态机设计
typedef enum logic [1:0] {IDLE, WORK, FINAL} fsm_state;
fsm_state fsm_state_reg;

//计数到LEN需要多少位二进制计数器
```

```

localparam CNT_LEN = $clog2(LEN);

// 迭代需要的立即数, 控制乘法循环
logic [CNT_LEN-1:0] work_cnt;

reg finish_reg;
assign product = product_reg;
assign finish = finish_reg;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        fsm_state_reg <= IDLE;
    end else begin
        case (fsm_state_reg)
            IDLE: begin
                finish_reg <= 1'b0;
                if(start) begin
                    multiplicand_reg <= multiplicand;
                    product_reg[PRODUCT_LEN-1:LEN] <= {LEN{1'b0}};
                    product_reg[LEN-1:0] <= multiplier;
                    work_cnt <= 0;
                    fsm_state_reg <= WORK;
                end else begin
                    fsm_state_reg <= IDLE;
                end
            end
            WORK: begin
                if(product_reg[0] == 1'b1) begin
                    product_reg <= ({1'b0, product_reg[PRODUCT_LEN-1:LEN]}
+ multiplicand_reg), product_reg[LEN-1:1]);
                end else begin
                    product_reg <= {1'b0, product_reg[PRODUCT_LEN-1:1]};
                end
                work_cnt <= work_cnt + 1;

                if(work_cnt == CNT_NUM[CNT_LEN-1:0]) begin
                    fsm_state_reg <= FINAL;
                end else begin
                    fsm_state_reg <= WORK;
                end
            end
        endcase
    end
end

```

```
        end
        FINAL: begin
            finish_reg <= 1'b1;
            fsm_state_reg <= IDLE;
        end
        default : fsm_state_reg <= IDLE;
    endcase
end
end
endmodule
```